Case_Study-Bike_Share_Analysis

Yesha Chokshi

2026-01-07

Data Preparation, Analysis and Visualization

This document contains the step by step process to recreate the steps conducted to prepapre this analysis.

Data Preparation

Step 1: Import Data into RStudio Load the library in your session (needs to be done every new session)

```
library(tidyverse)
```

```
## -- Attaching core tidyverse packages ------------------------ tidyverse 2.0.0 --
## v dplyr     1.1.4     v readr     2.1.6
## v forcats   1.0.1     v stringr   1.6.0
## v ggplot2   4.0.1     v tibble    3.3.0
## v lubridate 1.9.4     v tidyr     1.3.2
## v purrr     1.2.0
## -- Conflicts ------------------------------------------ tidyverse_conflicts() --
## x dplyr::filter() masks stats::filter()
## x dplyr::lag()    masks stats::lag()
## i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to become errors
```

```
library(DescTools)
```

Use read_csv to import the data - note this will change based on where the file is located

```
cyclist_2019_raw <- read_csv("Divvy_Trips_2019_Q1.csv")
```

```
## Rows: 365069 Columns: 12
## -- Column specification --------------------------------------------------------
## Delimiter: ","
## chr (6): start_time, end_time, from_station_name, to_station_name, usertype,...
## dbl (5): trip_id, bikeid, from_station_id, to_station_id, birthyear
## num (1): tripduration
##
## i Use `spec()` to retrieve the full column specification for this data.
## i Specify the column types or set `show_col_types = FALSE` to quiet this message.
```

```
cyclist_2020_raw <- read_csv("Divvy_Trips_2020_Q1.csv")
```

```
## Rows: 426887 Columns: 13
## -- Column specification --------------------------------------------------------
## Delimiter: ","
## chr (7): ride_id, rideable_type, started_at, ended_at, start_station_name, e...
## dbl (6): start_station_id, end_station_id, start_lat, start_lng, end_lat, en...
##
```

```
## i Use `spec()` to retrieve the full column specification for this data.
## i Specify the column types or set `show_col_types = FALSE` to quiet this message.
```

```
head(cyclist_2019_raw)
```

```
## # A tibble: 6 x 12
##    trip_id start_time         end_time       bikeid tripduration from_station_id
##      <dbl> <chr>              <chr>           <dbl>        <dbl>           <dbl>
## 1 21742443 2019-01-01 0:04:37 2019-01-01 0:~   2167          390             199
## 2 21742444 2019-01-01 0:08:13 2019-01-01 0:~   4386          441              44
## 3 21742445 2019-01-01 0:13:23 2019-01-01 0:~   1524          829              15
## 4 21742446 2019-01-01 0:13:45 2019-01-01 0:~    252         1783             123
## 5 21742447 2019-01-01 0:14:52 2019-01-01 0:~   1170          364             173
## 6 21742448 2019-01-01 0:15:33 2019-01-01 0:~   2437          216              98
## # i 6 more variables: from_station_name <chr>, to_station_id <dbl>,
## #   to_station_name <chr>, usertype <chr>, gender <chr>, birthyear <dbl>
```

```
head(cyclist_2020_raw)
```

```
## # A tibble: 6 x 13
##   ride_id  rideable_type started_at ended_at start_station_name start_station_id
##   <chr>    <chr>         <chr>      <chr>    <chr>                          <dbl>
## 1 EACB191~ docked_bike   2020-01-2~ 2020-01~ Western Ave & Lel~               239
## 2 8FED874~ docked_bike   2020-01-3~ 2020-01~ Clark St & Montro~               234
## 3 789F3C2~ docked_bike   2020-01-0~ 2020-01~ Broadway & Belmon~               296
## 4 C9A388D~ docked_bike   2020-01-0~ 2020-01~ Clark St & Randol~                51
## 5 943BC3C~ docked_bike   2020-01-3~ 2020-01~ Clinton St & Lake~                66
## 6 6D9C8A6~ docked_bike   2020-01-1~ 2020-01~ Wells St & Hubbar~               212
## # i 7 more variables: end_station_name <chr>, end_station_id <dbl>,
## #   start_lat <dbl>, start_lng <dbl>, end_lat <dbl>, end_lng <dbl>,
## #   member_casual <chr>
```

Step 2: Align Columns Amongst Dataframes Synchronize column names to match 2020 data if exists

```
cyclist_2019_renamed <- rename(cyclist_2019_raw, ride_id = trip_id,
                               started_at = start_time,
                               ended_at = end_time,
                               start_station_name = from_station_name,
                               end_station_name = to_station_name,
                               start_station_id = from_station_id,
                               end_station_id = to_station_id,
                               member_casual = usertype)

head(cyclist_2019_renamed)
```

```
## # A tibble: 6 x 12
##    ride_id started_at         ended_at      bikeid tripduration start_station_id
##      <dbl> <chr>              <chr>          <dbl>        <dbl>            <dbl>
## 1 21742443 2019-01-01 0:04:37 2019-01-01 0~   2167          390              199
## 2 21742444 2019-01-01 0:08:13 2019-01-01 0~   4386          441               44
## 3 21742445 2019-01-01 0:13:23 2019-01-01 0~   1524          829               15
## 4 21742446 2019-01-01 0:13:45 2019-01-01 0~    252         1783              123
## 5 21742447 2019-01-01 0:14:52 2019-01-01 0~   1170          364              173
## 6 21742448 2019-01-01 0:15:33 2019-01-01 0~   2437          216               98
## # i 6 more variables: start_station_name <chr>, end_station_id <dbl>,
## #   end_station_name <chr>, member_casual <chr>, gender <chr>, birthyear <dbl>
```

Check unique values in member_casual column

```
unique(cyclist_2019_renamed$member_casual)
```

```
## [1] "Subscriber" "Customer"
```

```
unique(cyclist_2020_raw$member_casual)
```

```
## [1] "member" "casual"
```

Change data usertype data from subscriber/customer to member/casual in 2019 member_casual column

```
cyclist_2019_renamed$member_casual[cyclist_2019_renamed$member_casual
                                   == "Customer"] <- "casual"

cyclist_2019_renamed$member_casual[cyclist_2019_renamed$member_casual
                                   == "Subscriber"] <- "member"
head(cyclist_2019_renamed)
```

```
## # A tibble: 6 x 12
##    ride_id started_at         ended_at      bikeid tripduration start_station_id
##      <dbl> <chr>              <chr>          <dbl>        <dbl>            <dbl>
## 1 21742443 2019-01-01 0:04:37 2019-01-01 0~   2167          390              199
## 2 21742444 2019-01-01 0:08:13 2019-01-01 0~   4386          441               44
## 3 21742445 2019-01-01 0:13:23 2019-01-01 0~   1524          829               15
## 4 21742446 2019-01-01 0:13:45 2019-01-01 0~    252         1783              123
## 5 21742447 2019-01-01 0:14:52 2019-01-01 0~   1170          364              173
## 6 21742448 2019-01-01 0:15:33 2019-01-01 0~   2437          216               98
## # i 6 more variables: start_station_name <chr>, end_station_id <dbl>,
## #   end_station_name <chr>, member_casual <chr>, gender <chr>, birthyear <dbl>
```

Check data types of all columns

```
str(cyclist_2019_renamed)
```

```
## spc_tbl_ [365,069 x 12] (S3: spec_tbl_df/tbl_df/tbl/data.frame)
##  $ ride_id           : num [1:365069] 21742443 21742444 21742445 21742446 21742447 ...
##  $ started_at        : chr [1:365069] "2019-01-01 0:04:37" "2019-01-01 0:08:13" "2019-01-01 0:13:23"
##  $ ended_at          : chr [1:365069] "2019-01-01 0:11:07" "2019-01-01 0:15:34" "2019-01-01 0:27:12"
##  $ bikeid            : num [1:365069] 2167 4386 1524 252 1170 ...
##  $ tripduration      : num [1:365069] 390 441 829 1783 364 ...
##  $ start_station_id  : num [1:365069] 199 44 15 123 173 98 98 211 150 268 ...
##  $ start_station_name: chr [1:365069] "Wabash Ave & Grand Ave" "State St & Randolph St" "Racine Ave &
##  $ end_station_id    : num [1:365069] 84 624 644 176 35 49 49 142 148 141 ...
##  $ end_station_name  : chr [1:365069] "Milwaukee Ave & Grand Ave" "Dearborn St & Van Buren St (*)" "W
##  $ member_casual     : chr [1:365069] "member" "member" "member" "member" ...
##  $ gender            : chr [1:365069] "Male" "Female" "Female" "Male" ...
##  $ birthyear         : num [1:365069] 1989 1990 1994 1993 1994 ...
##  - attr(*, "spec")=
##   .. cols(
##   ..   trip_id = col_double(),
##   ..   start_time = col_character(),
##   ..   end_time = col_character(),
##   ..   bikeid = col_double(),
##   ..   tripduration = col_number(),
##   ..   from_station_id = col_double(),
##   ..   from_station_name = col_character(),
##   ..   to_station_id = col_double(),
```

```
##   ..   to_station_name = col_character(),
##   ..   usertype = col_character(),
##   ..   gender = col_character(),
##   ..   birthyear = col_double()
##   .. )
##  - attr(*, "problems")=<externalptr>
```

```
str(cyclist_2020_raw)
```

```
## spc_tbl_ [426,887 x 13] (S3: spec_tbl_df/tbl_df/tbl/data.frame)
##  $ ride_id           : chr [1:426887] "EACB19130B0CDA4A" "8FED874C809DC021" "789F3C21E472CA96" "C9A38
##  $ rideable_type     : chr [1:426887] "docked_bike" "docked_bike" "docked_bike" "docked_bike" ...
##  $ started_at        : chr [1:426887] "2020-01-21 20:06:59" "2020-01-30 14:22:39" "2020-01-09 19:29:1
##  $ ended_at          : chr [1:426887] "2020-01-21 20:14:30" "2020-01-30 14:26:22" "2020-01-09 19:32:1
##  $ start_station_name: chr [1:426887] "Western Ave & Leland Ave" "Clark St & Montrose Ave" "Broadway
##  $ start_station_id  : num [1:426887] 239 234 296 51 66 212 96 96 212 38 ...
##  $ end_station_name  : chr [1:426887] "Clark St & Leland Ave" "Southport Ave & Irving Park Rd" "Wilte
##  $ end_station_id    : num [1:426887] 326 318 117 24 212 96 212 212 96 100 ...
##  $ start_lat         : num [1:426887] 42 42 41.9 41.9 41.9 ...
##  $ start_lng         : num [1:426887] -87.7 -87.7 -87.6 -87.6 -87.6 ...
##  $ end_lat           : num [1:426887] 42 42 41.9 41.9 41.9 ...
##  $ end_lng           : num [1:426887] -87.7 -87.7 -87.7 -87.6 -87.6 ...
##  $ member_casual     : chr [1:426887] "member" "member" "member" "member" ...
##  - attr(*, "spec")=
##   .. cols(
##   ..   ride_id = col_character(),
##   ..   rideable_type = col_character(),
##   ..   started_at = col_character(),
##   ..   ended_at = col_character(),
##   ..   start_station_name = col_character(),
##   ..   start_station_id = col_double(),
##   ..   end_station_name = col_character(),
##   ..   end_station_id = col_double(),
##   ..   start_lat = col_double(),
##   ..   start_lng = col_double(),
##   ..   end_lat = col_double(),
##   ..   end_lng = col_double(),
##   ..   member_casual = col_character()
##   .. )
##  - attr(*, "problems")=<externalptr>
```

Change data type for ride_id 2019 from numeric to character

```
cyclist_2019_renamed$ride_id <- as.character(cyclist_2019_renamed$ride_id)
```

Step 3: Join Dataframes Outer join two dataframes into one

```
cyclist_full_joined <- full_join(cyclist_2019_renamed, cyclist_2020_raw,
                           by = c("ride_id", "started_at", "ended_at",
                           "start_station_name", "end_station_name",
                           "start_station_id", "end_station_id",
                           "member_casual"))
head(cyclist_full_joined)
```

```
## # A tibble: 6 x 17
##   ride_id  started_at         ended_at      bikeid tripduration start_station_id
```

```
##   <chr>    <chr>               <chr>          <dbl>        <dbl>            <dbl>
## 1 21742443 2019-01-01 0:04:37 2019-01-01 0~   2167          390              199
## 2 21742444 2019-01-01 0:08:13 2019-01-01 0~   4386          441               44
## 3 21742445 2019-01-01 0:13:23 2019-01-01 0~   1524          829               15
## 4 21742446 2019-01-01 0:13:45 2019-01-01 0~    252         1783              123
## 5 21742447 2019-01-01 0:14:52 2019-01-01 0~   1170          364              173
## 6 21742448 2019-01-01 0:15:33 2019-01-01 0~   2437          216               98
## # i 11 more variables: start_station_name <chr>, end_station_id <dbl>,
## #   end_station_name <chr>, member_casual <chr>, gender <chr>, birthyear <dbl>,
## #   rideable_type <chr>, start_lat <dbl>, start_lng <dbl>, end_lat <dbl>,
## #   end_lng <dbl>
```

Step 4: Sort and Filter Dataframes Sort data from oldest to newest start time and then end time

```
cyclist_cleansed <- arrange(cyclist_full_joined, started_at, ended_at)
```

Remove columns not present in the orginal two dataframes

```
cyclist_cleansed <- select(cyclist_cleansed, -bikeid, -tripduration, -gender,
                           -birthyear, -rideable_type, -start_lat, -start_lng,
                           -end_lat, -end_lng)
head(cyclist_cleansed)
```

```
## # A tibble: 6 x 8
##   ride_id started_at ended_at start_station_id start_station_name end_station_id
##   <chr>   <chr>      <chr>               <dbl> <chr>                       <dbl>
## 1 217424~ 2019-01-0~ 2019-01~              199 Wabash Ave & Gran~             84
## 2 217424~ 2019-01-0~ 2019-01~               44 State St & Randol~            624
## 3 217424~ 2019-01-0~ 2019-01~               15 Racine Ave & 18th~            644
## 4 217424~ 2019-01-0~ 2019-01~              123 California Ave & ~            176
## 5 217424~ 2019-01-0~ 2019-01~              173 Mies van der Rohe~             35
## 6 217424~ 2019-01-0~ 2019-01~               98 LaSalle St & Wash~             49
## # i 2 more variables: end_station_name <chr>, member_casual <chr>
```

Check for any blank spaces

```
colSums(is.na(cyclist_cleansed)) #which columns have blanks
```

```
##            ride_id         started_at           ended_at   start_station_id
##                  0                  0                  0                  0
## start_station_name     end_station_id   end_station_name      member_casual
##                  0                  1                  1                  0
```

```
which(is.na(cyclist_cleansed), arr.ind=TRUE) #which row has blanks in this case row 755166
```

```
##         row col
## [1,] 755166   6
## [2,] 755166   7
```

Check blank row to make sure this can be removed

```
check_row <- cyclist_cleansed %>%
  slice(755166)
```

Remove blank row

```
cyclist_cleansed <- na.omit(cyclist_cleansed)
```

Add blank ride_length and day_of_week columns into cleansed Dataframe

```
cyclist_cleansed <- cyclist_cleansed %>%
  mutate(ride_length = NA, day_of_week = NA)
```

Calculate ride length and day of week in cleansed table

```
cyclist_cleansed$ride_length <- as.numeric(difftime(cyclist_cleansed$ended_at,
                                                    cyclist_cleansed$started_at,
                                                    units = "secs"))
cyclist_cleansed$day_of_week <- weekdays(as.Date
                                         (cyclist_cleansed$started_at))
head(cyclist_cleansed)
```

```
## # A tibble: 6 x 10
##   ride_id started_at ended_at start_station_id start_station_name end_station_id
##   <chr>   <chr>      <chr>               <dbl> <chr>                        <dbl>
## 1 217424~ 2019-01-0~ 2019-01~              199 Wabash Ave & Gran~             84
## 2 217424~ 2019-01-0~ 2019-01~               44 State St & Randol~            624
## 3 217424~ 2019-01-0~ 2019-01~               15 Racine Ave & 18th~            644
## 4 217424~ 2019-01-0~ 2019-01~              123 California Ave & ~            176
## 5 217424~ 2019-01-0~ 2019-01~              173 Mies van der Rohe~             35
## 6 217424~ 2019-01-0~ 2019-01~               98 LaSalle St & Wash~             49
## # i 4 more variables: end_station_name <chr>, member_casual <chr>,
## #   ride_length <dbl>, day_of_week <chr>
```

Filter out any ride length over 24 hours (86400 seconds)

```
cyclist_filtered <- cyclist_cleansed[cyclist_cleansed$ride_length
                                     <= 86400, ]
```

Rename 'member' member type to annual

```
cyclist_filtered$member_casual[cyclist_filtered$member_casual
                               == "member"] <- "annual"
head(cyclist_filtered)
```

```
## # A tibble: 6 x 10
##   ride_id started_at ended_at start_station_id start_station_name end_station_id
##   <chr>   <chr>      <chr>               <dbl> <chr>                        <dbl>
## 1 217424~ 2019-01-0~ 2019-01~              199 Wabash Ave & Gran~             84
## 2 217424~ 2019-01-0~ 2019-01~               44 State St & Randol~            624
## 3 217424~ 2019-01-0~ 2019-01~               15 Racine Ave & 18th~            644
## 4 217424~ 2019-01-0~ 2019-01~              123 California Ave & ~            176
## 5 217424~ 2019-01-0~ 2019-01~              173 Mies van der Rohe~             35
## 6 217424~ 2019-01-0~ 2019-01~               98 LaSalle St & Wash~             49
## # i 4 more variables: end_station_name <chr>, member_casual <chr>,
## #   ride_length <dbl>, day_of_week <chr>
```

Analysis

Calculate Mean of all ride_lengths without outliers

```
mean_ride_length <- mean(cyclist_filtered$ride_length)
max_ride_length <- max(cyclist_filtered$ride_length)
mode_ride_length <- Mode(cyclist_filtered$day_of_week)
```

Calculate mean of casual and member ride_lengths

```
means_by_member_type <- cyclist_filtered %>%
  group_by(day_of_week, member_casual) %>%
  summarise(mean_ride_length = mean(ride_length,
                                    na.rm = TRUE))
```

```
## `summarise()` has grouped output by 'day_of_week'. You can override using the
## `.groups` argument.
```

Calculate mean of ride_lengths for day of week

```
means_by_day_of_week <- cyclist_filtered %>%
  group_by(day_of_week) %>%
  summarise(mean_ride_length = mean(ride_length,
                                    na.rm = TRUE))
```

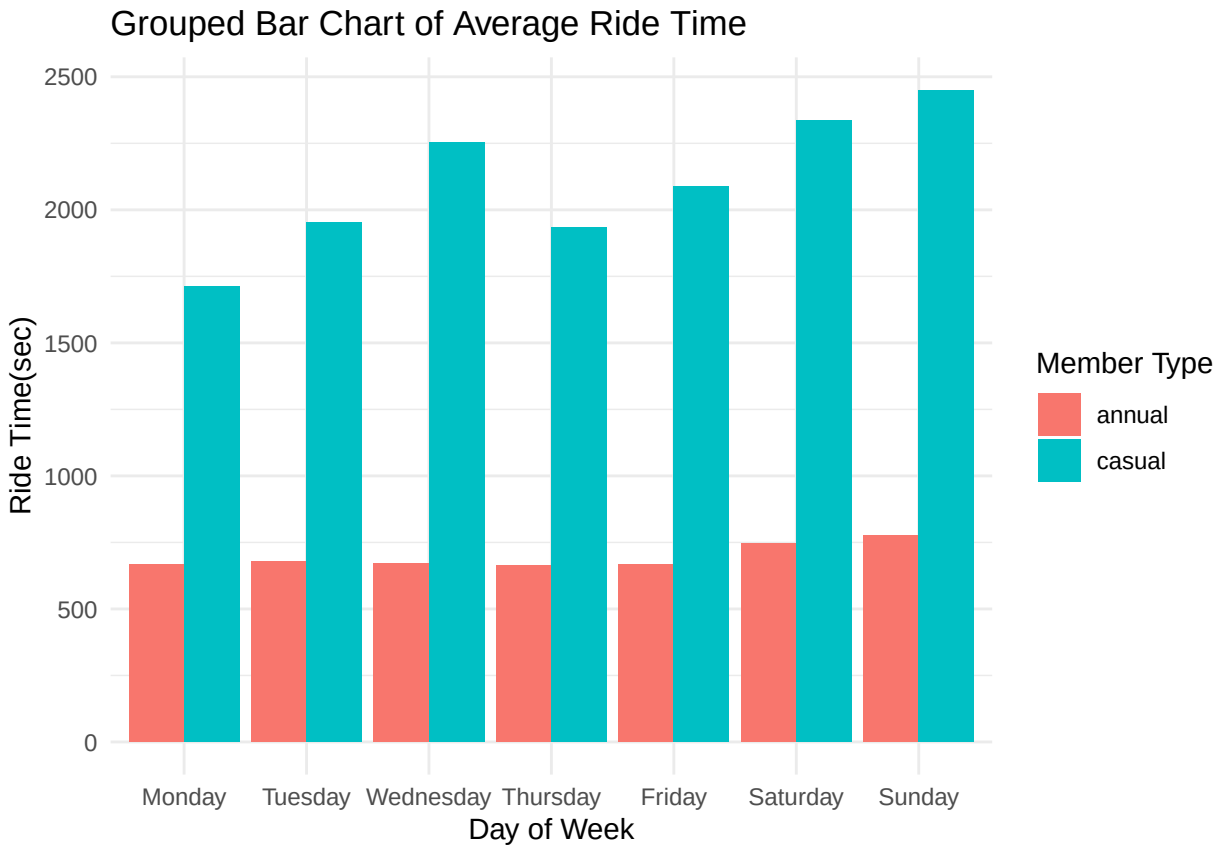
Order weekdays in chronological order for means by member type dataframe

```
day_order <- c("Monday", "Tuesday", "Wednesday", "Thursday", "Friday", "Saturday", "Sunday")

means_by_member_type$day_of_week <- factor(means_by_member_type$day_of_week,
                                           levels = day_order)

means_by_day_of_week$day_of_week <- factor(means_by_day_of_week$day_of_week,
                                           levels = day_order)
```

Data Analysis Vizualization

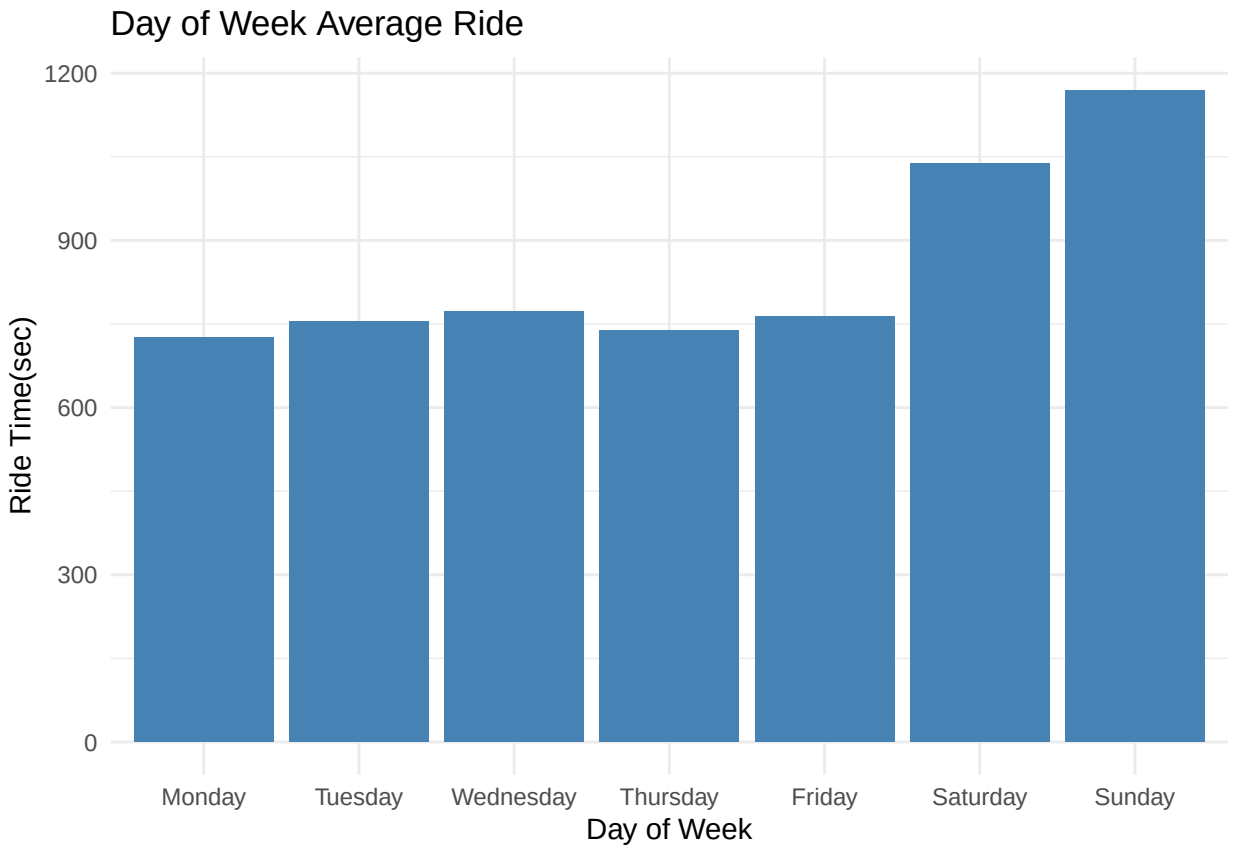
Bar Plot to visualize average ride lengths between member type per day

```
ggplot(means_by_member_type, aes(x = day_of_week,
                                 y = mean_ride_length, fill = member_casual)) +
  geom_bar(position = "dodge", stat = "identity") +
  labs(title = "Grouped Bar Chart of Average Ride Time",
       x = "Day of Week",
       y = "Ride Time(sec)",
       fill = "Member Type") +
  theme_minimal()
```



Bar Plot to visualize average ride lengths between member type per day

```
ggplot(means_by_day_of_week, aes(x = day_of_week, y = mean_ride_length)) +
  geom_bar(stat = "identity", fill = "steelblue") +
  labs(title = "Day of Week Average Ride",
       x = "Day of Week",
       y = "Ride Time(sec)") +
  theme_minimal()
```


Day of Week Average Ride
1200
900
600
300
0
Ride Time(sec)
Monday
Tuesday
Wednesday
Thursday
Friday
Saturday
Sunday
Day of Week